

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 748 — Computational Photography

What a camera measures, what a photograph is, and the computation that now sits between them

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 4 (with CSCE 650 and CSCE 735)

Prerequisites: CSCE 641 Computer Graphics; CSCE 647 helpful for the radiometry; linear algebra; comfort with signals and convolution, developed here as needed

Effort: 10–12 hours per week · 13 modules + 2 project weeks

Deliverable: An image pipeline you wrote: raw to display, HDR merge, alignment, gradient-domain compositing, and a burst denoiser — evaluated against ground truth you captured yourself

Course description

CSCE 647 computed images from a scene description. This course goes the other way: **it starts with measurements and asks what can be recovered from them**. The two are the same mathematics approached from opposite ends, and several results in this course are rendering results read backwards.

The organising fact is that **a camera does not capture a photograph; it captures measurements from which a photograph is computed**. That was always true — film had a response curve and a development process — and it has become overwhelmingly true. A modern phone photograph is the output of alignment, merging, demosaicing, tone mapping, and several learned models operating on a burst of underexposed frames. The button press is the start of the computation, not the end.

This has a consequence worth stating early: **the boundary between photography and synthesis has become difficult to locate**. A night-mode image shows detail no single exposure recorded; a portrait-mode image has a depth of field the lens never had; a learned upscaler produces texture that was never measured. Module 13 takes that seriously rather than treating it as a curiosity.

You will implement the pipeline. By the end you will be able to take a raw file and produce a displayable image through code you wrote, and explain every decision between the two.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain image formation and what a raw file actually contains.
- 2 Implement the core image operations: convolution, pyramids, and edge-aware filtering.
- 3 Recover a camera response curve and merge exposures into an HDR radiance map.
- 4 Implement tone mapping and explain why it is underdetermined.
- 5 Implement demosaicing and the raw-to-display pipeline.
- 6 Explain deconvolution and why blind deblurring is hard.
- 7 Align and composite images robustly.
- 8 Apply gradient-domain methods and explain why they work.
- 9 Explain light fields and the plenoptic function.
- 10 Implement a burst photography pipeline.
- 11 Assess learned image methods honestly, including what they invent.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	What a Camera Measures	What is actually in a raw file?
2	Image Processing Foundations	What are the primitives of image computation?
3	High Dynamic Range	How do you photograph a scene brighter than your sensor?
4	Tone Mapping	How do you compress dynamic range without destroying the image?
5	Demosaicing and the Raw Pipeline	How do you get a colour image from a sensor that measures one channel per pixel?
6	Deconvolution and Deblurring	Can you recover a sharp image from a blurred one?
7	Alignment and Stitching	How do you combine images taken from different positions?
8	Gradient-Domain Processing	Why does pasting in the gradient domain look seamless?
9	Light Fields and Plenoptic Cameras	What if you recorded direction as well as position?
10	Burst Photography	How does a small sensor produce a usable night photograph?
11	Depth, Matting, and Synthetic Defocus	How does portrait mode work, and when does it fail?
12	Learned Image Processing	When a model produces detail, where did it come from?

Wk	Module	Core question it answers
13	What a Photograph Is Now	If the image is computed, what does it document?

Projects

Project 1 — From raw to a displayed image (after Module 7)

The complete classical pipeline, written by you, on photographs you took. Everything afterwards builds on this, and the discipline of handling linear data correctly is the point.

Requirements

- A raw decoder path: black level, white balance, demosaic, colour matrix, and display encoding, each as a separate inspectable stage.
- A camera response curve recovered from a bracketed exposure sequence you captured.
- An HDR radiance map merged from those exposures, with weights justified.
- At least two tone mapping operators, one global and one local, implemented rather than called.
- A panorama from at least five hand-held images, with feature matching, RANSAC homography, and blending.
- Every intermediate stage saved and shown.

Done means

- **Your raw pipeline's output compared against the camera's own JPEG**, with the differences identified and explained rather than apologised for.
- A recovered response curve plotted against the exposures used to recover it, with the fit shown.
- **A tone mapping comparison on the same radiance map**, with an honest statement of what each operator destroyed.
- A panorama with no visible seams, plus the same panorama with naive blending for comparison.

Project 2 — A modern computational pipeline (after Module 12)

Something a phone does, implemented and measured. Choose one and do it properly rather than three partially.

Requirements

- **Either** a burst denoising pipeline — align, merge, and denoise a burst of underexposed raws; **or** a portrait-mode pipeline — depth estimation and physically-motivated synthetic defocus; **or** a gradient-domain compositing tool with seamless cloning and a usable interface.
- Quantitative evaluation against a ground truth you captured — a long exposure, a measured depth, or a reference composite.
- A failure analysis: the input on which your implementation produces a visibly wrong result.
- A comparison against a learned method if one is available, with an honest account of what it does better and what it invents.

Done means

- **A quantitative result against ground truth** — PSNR, SSIM, or a task-appropriate measure, with the measurement method stated.
- A figure showing the pipeline stage by stage on one input.
- **A documented failure with its mechanism explained.** Every pipeline has one.
- A written paragraph on which parts of your output were *measured* and which were *inferred*. **Module 13's question, answered for your own work.**

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
CMU 15-463 Computational Photography — Gkioulekas (free lectures, slides, assignments)	The primary source. Complete lecture notes and programming assignments covering most of this course, with unusually careful treatment of the physics.
Marc Levoy — Lectures on Digital Photography (free)	From the person who led Google's camera work and co-invented light field photography. Exceptional on optics, exposure, and why the phone pipeline looks the way it does.
Szeliski — Computer Vision: Algorithms and Applications, 2nd ed. (free online)	Chapters 3, 8, and 9 cover image processing, alignment, and stitching. The standard reference, free, and the alignment material for Module 07 is the best there is.
Georgia Tech CS6475 Computational Photography (free)	A second voice with more worked examples and a gentler pace. Good for Modules 03, 04, and 08.

Resource	Role
Reinhard et al. — High Dynamic Range Imaging	The standard HDR reference. Library copy; the key papers by the same authors are free and linked per module.
Hasinoff et al. — Burst photography for high dynamic range and low-light imaging (free)	The HDR+ paper, with a full dataset of raw bursts. This is what your phone does, documented by the people who built it.

Tooling

- **Python with NumPy** for everything except the inner loops. This is a course about algorithms on arrays and the language should not be in the way.
- **No OpenCV for the core assignments.** Convolution, demosaicing, alignment, and blending are the course; calling a library function for them teaches nothing. Use it for I/O and for checking.
- **rawpy / LibRaw** to read raw files, and **a camera that shoots raw** — a phone with a raw mode is sufficient and many have one.
- **OpenEXR** for high dynamic range intermediates. Writing 8-bit PNG between stages destroys exactly what this course is about.
- **A tripod**, or a stable surface. Several assignments need aligned exposures, and hand-held alignment is Module 07's problem rather than Module 03's.
- **A colour chart** if you can get one — even a printed one helps for white balance and response curve work.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.